

AUIGridReact.tsx 서브 컴포넌트 설명서

AUIGridReact.tsx 는 Typescript + React.js 라이브러리에서 AUIGrid 를 쉽게 사용할 수 있도록 작성한 컴포넌트입니다.

1. 정의 가능한 속성들

속성명	유형(Type)	기본값	설명
autoResize	Boolean	true	resizeMode에 따른 자동 리사이징 사용 여부입니다. false이면 감지를 등록하지 않습니다.
columnLayout	IGrid.Column[]	[]	그리드 칼럼 레이아웃을 지정합니다.
footerLayout	IGrid.Footer[]	[]	그리드 푸터 레이아웃을 지정합니다.
gridProps	IGrid.Props	{}	그리드 속성을 지정합니다.
name	String	''	그리드 이름을 지정합니다.
resizeDelayTime	Number	300	마지막 크기 변경 알림 후 기다리는 시간(ms)입니다. 두 감지 방식에 모두 적용됩니다.
resizeMode	String	"window"	"window"는 창 크기 이벤트, "container"는 호스트 DIV의 크기 변화를 감지합니다. 4절을 참고하십시오.
waitPortalRendering	Boolean	false	Modal이나 Teleport 등 포탈 방식 렌더링을 위해 RAF 사용으로 생성할지 여부를 지정합니다.

* name 속성은 의무사항이 아닙니다. 그러나 name 을 지정하면 해당 name 값을 id 로 갖는 그리드가 생성됩니다. 예로 name='showcase01' 로 설정했다면 해당 그리드는 'aui-grid-wrap-showcase01' 을 id 로 갖는 그리드로 생성됩니다.

DOM 요소를 확인하면 다음과 같이 그리드 생성 div의 id 를 확인 할 수 있습니다.

```
<div id="aui-grid-wrap-showcase01" style="position: relative;"> == $0
  <div class="aui-grid" style="position: relative; box-sizing: content-box"
    </div>
```

단, 서버사이드 렌더링(SSR)을 하는 프로젝트나 AUIGridReact.tsx 가 동적으로 생성/제거되는 경우에 name은 고유값 설정이 의무화 됩니다.

2. 그리드에 접근하여 메소드 사용하기

React.js 에서 제공하는 useRef Hook 을 이용하여 다음처럼 ref 를 참조하도록 합니다.

```
// 그리드 객체
const myGrid = useRef<AUIGrid>(null);
```

위와 같이 useRef 로 myGrid 를 생성한 후 AUIGrid 생성 태그에 ref 로 설정합니다.

```
<AUIGrid ref={myGrid} />
```

그러면 언제든지 리액트 자신의 스코프에서 다음처럼 그리드에 접근 할 수 있습니다.

```
const grid = myGrid.current as AUIGrid;
grid.addRow();
```

3. 리액트에서 AUIGridReact.tsx 사용 예시

```
import React, { useEffect, useRef } from 'react';
import * as IGrid from 'aui-grid';
import AUIGrid from '../static/AUIGrid-React.tsx/AUIGridReact';
import axios from 'axios';

const SampleDefault = () => {
  // 그리드 객체
  const myGrid = useRef<AUIGrid>(null);

  // 그리드 칼럼 레이아웃 정의
  const columnLayout: IGrid.Column[] = [
    {
      dataField: 'id',
      headerText: 'ID',
      width: 120,
    },
    {
      dataField: 'name',
      headerText: 'Name',
      width: 140,
    },
    {
      dataField: 'country',
      headerText: 'Country',
      width: 140,
    },
    {
      dataField: 'price',
      headerText: 'Price',
    }
  ]
}
```

```

        dataType: 'numeric',
        style: 'my-column',
        width: 120,
    },
    {
        dataField: 'quantity',
        headerText: 'Quantity',
        dataType: 'numeric',
        style: 'my-column',
        width: 100,
    },
    {
        dataField: 'date',
        headerText: 'Date',
        dataType: 'date',
    },
],
];

// 그리드 속성 정의
const gridProps: IGrid.Props = {
    width: '100%',
    height: 480,
    // 편집 가능 여부 (기본값 : false)
    editable: true,
    noDataMessage: '출력할 데이터가 없습니다.',
    groupingMessage: '여기에 칼럼을 드래그하면 그룹핑이 됩니다.',
};

useEffect(() => {
    console.log('SampleDefault 마운트됨');

    // 최초 마운팅 될 때 그리드 이벤트 세팅
    setupGridEvents();

    // 최초 마운팅 될 때 그리드 데이터 조회시키기
    requestGridData();

    return () => {
        console.log('SampleDefault 언마운트됨');
    };
}, []);

// 그리드 이벤트 세팅
const setupGridEvents = () => {
    const grid = myGrid.current as AUIGrid;

    // 그리드 ready 이벤트 바인딩

```

```

        grid.bind(IGrid.EventKind.Ready, (event: IGrid.ReadyEvent) => {
            console.log(event);
        });

        // 그리드 cellClick, headerClick 이벤트 바인딩
        grid.bind([IGrid.EventKind.CellClick, IGrid.EventKind.HeaderClick],
(event: IGrid.CellClickEvent | IGrid.HeaderClickEvent) => {
            console.log(event);
        });
    };

    // 그리드 데이터 조회하여 삽입
    const requestGridData = () => {
        const grid = myGrid.current as AUIGrid;
        grid.showAjaxLoader();
        axios.get('./data/normal_100.json').then((result) => {
            //console.log(result);
            // 그리드 데이터 삽입
            grid.setGridData(result.data);
            grid.removeAjaxLoader();
        });
    };

    return (
        <div>
            <AUIGrid ref={myGrid} columnLayout={columnLayout}
gridProps={gridProps} />
        </div>
    );
};

export default SampleDefault;

```

4 컨테이너 크기 감지 사용하기

resizeMode="container"를 지정하면 ResizeObserver로 그리드를 담는 호스트 DIV의 크기를 감지합니다. 창 크기가 그대로여도 사이드바 접기나 부모 레이아웃 변경으로 호스트 크기가 바뀌면 그리드를 조정합니다.

설정 예제

다음 옵션은 gridProps 내부가 아닌 AUIGrid 서브 컴포넌트에 직접 지정합니다. columnLayout과 gridProps는 앞 절에서 정의한 값을 사용합니다.

```
{/* 컨테이너 크기 변화에 따라 100ms 지연 후 조정합니다. */}
```

```
<AUIGrid
  ref={myGrid}
  resizeMode="container"
  resizeDelayTime={100}
  columnLayout={columnLayout}
  gridProps={gridProps}
/>
```

기본값과 호출 시점

resizeMode의 기본값은 "window"입니다. 생략하면 기존 창 크기 이벤트 방식을 유지합니다. autoResize의 기본값은 true이며, autoResize={false}로 지정하면 자동 감지를 등록하지 않습니다.

resizeDelayTime의 기본값은 300ms입니다. "container"에서는 마지막 알림 후 지정 시간만큼 기다리고 다음 애니메이션 프레임에 크기를 반영합니다. 0으로 설정해도 동기 실행하지 않습니다. 크기가 같거나 DOM이 분리되어 있거나 너비 또는 높이가 5px 이하이면 호출하지 않습니다.

resizeMode와 autoResize는 생성 시 지정하십시오. 실행 중 속성 변경만으로 감지 방식을 전환하지 않습니다. ResizeObserver가 없는 환경에서는 창 크기 이벤트로 대체합니다.

레이아웃과 생명주기

관찰 대상은 엔진 내부 셀이 아닌 호스트 DIV입니다. 앞 절처럼 width: "100%", height: 480을 사용하면 부모 너비에 맞추고 높이는 유지합니다. 높이도 부모에 맞추려면 호스트 CSS와 부모 높이를 지정하십시오. Flex/Grid에서는 필요에 따라 min-width: 0을 지정하고, 콘텐츠가 부모를 계속 늘리는 순환 배치를 피하십시오.

컴포넌트가 언마운트되거나 래퍼의 destroy()를 호출하면 관찰자와 예약된 타이머 및 애니메이션 프레임을 정리합니다. create()로 다시 만들면 관찰을 재연결합니다. 그리드 데이터와 상태를 리사이징을 위해 재생성하지 않습니다.

실제 resize()가 실행되면 편집 완료 등 엔진의 기존 리사이징 동작이 적용됩니다. 수동 resize() 사용법과 autoGridHeight의 데이터 기반 높이 정책은 유지됩니다.

- 감사합니다. -